Joint Computation Offloading and Data Caching in Multi-Access Edge Computing Enabled Internet of Vehicles

Liqing Liu, Xiaoming Yuan, *Member, IEEE*, Ning Zhang, *Senior Member, IEEE*, Decheng Chen, Keping Yu, *Member, IEEE*, and Amir Taherkordi

***Abstract*—Internet of Vehicles (IoV) has attracted global research interests across extensive applications. Due to the significant increase in the number of vehicles accessing the Internet, there are several challenges in designing efficient task offloading and data caching strategies to improve the utilization of the network resource and provide the users with high-quality services. To this end, this study proposes the task offloading and resource allocation schemes, including the selection of execution mode, data transmission path, the assignment of the sub-channels, the strategies of caching and caching updating in a Multi-Access Edge Computing (MEC) enabled IoV system with multiple mobile vehicles equipped with the capacity of energy harvesting. Specifically, the downlink relevant data or the uplink offloaded data can be transmitted through either the Macro Base Station(MBS) or the Road Side Unit(RSU). Also, we consider two different situations: off-peak hours and peak hours, in which the execution mode is different. In off-peak hours, the tasks can directly offload to the MEC server, and the average execution delay minimization problem is modelled as an integer programming problem, which is solved by Simulated Annealing Genetic Algorithm (SAGA). In peak hours, the tasks can be either executed locally or offloaded to the MEC server, and the formulated problem are more complicated, which are solved by Deep Q Network (DQN). Finally, a series of simulations are conducted to demonstrate the efficiency of the proposed schemes.**

***Index Terms*—MEC enabled IoV, task offloading, local execution, caching strategy, caching updating strategy.**

Manuscript received 2 October 2022; revised 28 February 2023; accepted 24 May 2023. Date of publication 12 June 2023; date of current version 14 November 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62201135, in part by the Natural Science Foundation of Hebei Province under Grant F2021501032, in part by the Science and Technology Project of Hebei Province Education Department under Grants QN2021307 and ZD2022164, in part by the Fundamental Research Funds for the Central Universities under Grant N2223031, and in part by the Open Research Project of Key Laboratory of Cognitive Radio and Information Processing, Ministry of Education, Guilin University of Electronic Technology) under Grant CRKL210203. The review of this article was coordinated by Prof. Xu Chen. *(Corresponding authors: Xiaoming Yuan; Keping Yu.)*

Liqing Liu, Xiaoming Yuan, and Decheng Chen are with the Qinhuangdao Branch Campus, Northeastern University, Qinhuangdao 066004, China (e-mail: liuliqing_yanyan@163.com; yuanxiaoming@neuq.edu.cn; 1170313019@qq.com).

Ning Zhang is with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON N9B 3P4, Canada (e-mail: ning.zhang@uwindsor.ca).

Keping Yu is with the Graduate School of Science and Engineering, Hosei University, Tokyo 184-8584, Japan (e-mail: keping.yu@ieee.org).

Amir Taherkordi is with the Department of Informatics, University of Oslo, 0316 Oslo, Norway (e-mail: amirhost@ifi.uio.no).

Digital Object Identifier 10.1109/TVT.2023.3285073

I. Introduction

With the assistance of the advanced mobile technologies and high data rate wireless networks, the Internet of Vehicles (IoV) realizes omnidirectional network integration for Vehicle-to-Everything(V2X). In addition, the vehicular application have exploded in recent years. These all promote the overall vehicular intelligent driving level, provide passengers with comfortable and intelligent driving experience [1], [2], [3]. However, it has to say that vehicles have limited computational and storage resources, which is hard to deal with the complicated applications with stringent delay and energy constraints [4].

Mobile Edge Computing (MEC) emerges as an new paradigm and computation offloading in MEC proves to be the potential solution to address the above challenges [5], [6], [7]. Also, computation offloading is one of the important technical support for UAV, which has attracted global research interests in academic circles. Moreover, the Macro Base Station (MBS) or the Road Side Units (RSUs) are equipped with edge servers with computing and storage resources, which can serve passing vehicles within their coverage and mitigate the central cloud's computing and backhaul pressures [8], [9]. But, it is worth mentioning that the computational and storage resource in the edge server can only serve a very small number of vehicles, compared with the central cloud. [10]

From the perspective of computation offloading mode, it can be performed in three different ways: binary offloading, partial offloading, and stochastic offloading [11], [12]. From the perspective of computation offloading environment, it can be divided into static computation offloading and dynamic computation, where the static offloading is constructed before the execution while the dynamic offloading adjusts execution strategy in real-time [13], [14], [15]. Dynamic computation offloading can improve the rate of resource utilization to make the system obtain a high quality of service (Qos). So in this article, we will consider a binary offloading in a dynamic network environment, i.e., binary dynamic offloading [16], [17], [18].

But it is a common phenomenon that a amount of unnecessary data transmission and request computation exist in the limited network, which is a terrible waste in the aspect of wireless and computation resources, as it often happens that offloading and processing the same tasks repeatedly. So it is an excellent solution to add a shareable cache space at the edge server. In this way,

0018-9545 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

the identical requests can directly obtain the computation results, which reduce the frequency of repeated data transmissions and computation [19], [20]. In addition, computation offloading and content caching are interrelated and coupled with each other. How to ensure the quality of computation offloading through the decision of content caching has long been considered as a challenge. Moreover, apart from the caching in the edge server, the vehicles are also can be regarded as collaborative caching agents. Based on the different characteristics between the edge server and vehicles, it is essential to design the joint caching optimization strategy to ensure the quality of service (QoS) of computation offloading [21].

With the progress of science and technology, it is a trend that electric vehicles are gradually replacing the market of gasoline vehicles. Energy harvesting as a green energy technology can further enable vehicles to harvest energy from various sources such as solar energy or electric energy [22], [23]. There is no doubt that the offloading decisions and resource allocation schemes have a significant impact on driving experiences of vehicles. Because of the limitation of storage space in MEC servers, the uncertainty of wireless channels, the asymmetry of traffic flow, the unknown of energy harvesting amount, *etc.*, dynamic computation offloading and resource optimization are challenging. In this research, the goal is to address those challenges by developing efficient dynamic computation offloading and resource allocation schemes with energy harvesting enabled vehicular networks.

In a summary, the main contributions of this article are as follows, which is also the different between previous work and their work.

1) We construct a hierarchical MEC enabled IoV network with multiple vehicles and jointly investigate the aspects of execution mode (either local execution or offloading), transmission path selection (either the MBS or the RSU with backhaul), subchannel assignment, and caching management. Also, the vehicles have equipped with the capability of energy harvesting, which make the parameters to be optimized have coupling relationship.
2) From the perspective of the IoV, we formulate the average delay minimization problem under the situations of off-peak hours and peak hours but the execution strategies are difficult. In the case of off-peak hours, the requests can directly be offloaded to the MEC server through either the MBS or the RSU without local execution; while in the case of peak hours, the requests can be either executed locally or be offloaded to the MEC server.
3) In addition, to solve the formulated problems, we adopt dynamic online algorithms such as SAGA and DQN, which can achieve the imitation learning efficiently.

The rest of the article is organized as follows. In Section II, we review the related work in recent years. In Section III, we present a MEC enabled IoV network, including the network model, the request model, the energy harvesting, and the caching model. In Section IV, we analyze the requests execution process, which consists of the local execution process and the offloading processing. In Section V, we formulate the two problems under the situations of off-peak hours and peak hours. In Section VI, we propose the SAGA algorithm and the DQN algorithm to solve the corresponding problem. In Section VII, we conduct simulations to demonstrate the effectiveness of the proposed schemes. In Section VIII, we conclude the article and discuss the future research direction.

II. Related Work

In this section, we review existing works on computation offloading, resource allocation and data caching in IoVs. The literature focus on the computation offloading to the cloud server without considering caching in the earlier period. For example, considering vehicle movement and parking status, Lee et al. [24] adopted a heuristic algorithm to use the parked vehicles to cooperate with a conventional edge server to form a collaborative edge computing, with the objective to minimize the service delay of the total vehicle group. Based on the characteristics of mutual cooperation and resource sharing among multiple server, Luo et al. [25] jointly made optimal computation offloading decisions and flying orientation choices for multi-UAV cooperative target search by utilizing the concept of uncertainty to evaluate the search process and DRL technique. Qi et al. [26] proposed a twofold computation offloading mechanism in a collaborative way for a multi-UAV assisted MEC network. Alam et al. [27] proposed a novel path selection scheme based on channel greedy scheduling algorithm and node buffer state for delay sensitive requests. Yen et al. [28] introduced a two-tier system including vehicle and edge server to minimize the total execution cost, by optimizing the offloading decisions including the selection of offloading destination and offloading ratio. Wang et al. [29] used a Markov decision process to study the collaborative computation offloading problem of mobile devices and vehicles, and took vehicle users as mobile cloud to assist mobile devices during the offloading process. Besides, considering the fast change of network topology caused by the high-speed movement of the vehicle, the selected vehicle is allowed to directly transmit the computation task to the next vehicle directly. Liu et al. [30] put forward an online algorithm for UAV swarms to jointly optimize the computation offloading and multi-hop routing scheduling in the Edge-Cloud environment.

With caching integrated into the computation offloading, the unnecessary data transmission can be reduced. In this way, the offloading efficiency and resource utilization are greatly improved. For example, in [31], an optimized problem was put forward based on the idea of pushing AI inspired computing, caching, and communication resources to the proximity of smart vehicles, and offloading the RSU peer and V2R as well as making content caching possible in the IoV framework. In [32], the authors proposed a novel QoE model and then formulated an enhanced deep reinforcement learning algorithm for an optimal offloading mode to reduce energy consumption. In [33], the authors presented a task-based architecture of content caching in VEC, which lures many researches to design the vast majority of technology that could boost efficiency. In [34], the authors proved the feasibility and necessity of the use of named data networking in vehicular environment based on content caching. In [35], a cooperative edge caching scheme was proposed,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

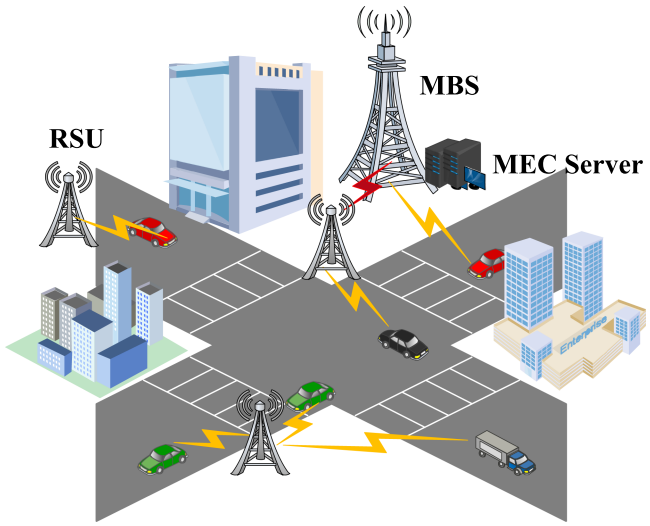


Fig. 1. The system model of a MEC enabled IoV network.

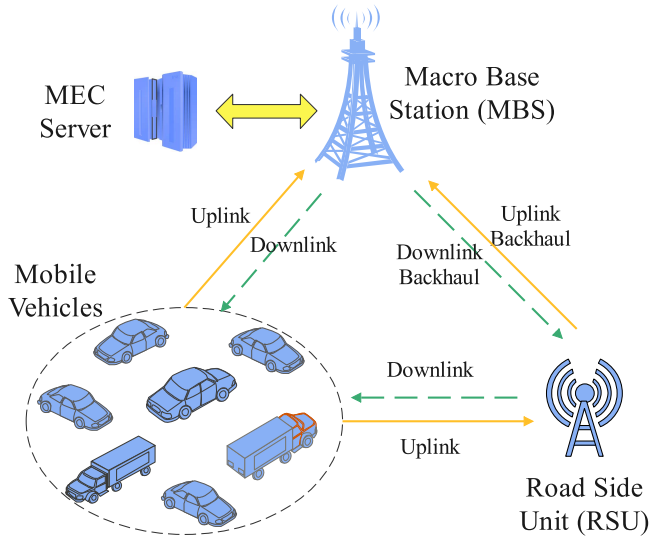


Fig. 2. The specific execution processing of the system.

aiming to optimize the placement and delivery of content in networks as well as the vehicular edge computing, by means of the flexible cooperation among a macro-cell station, roadside units, and smart vehicles. In [36], the authors propose a collaborative computation offloading and content caching method in edge computing for self-driving system and solved by DRL algorithm. In [37], with considering the cache resources competition, the authors formulate a joint optimization problem of content caching and cache-enhanced computation offloading in mobile edge service networks. In [38], the authors formulate a cache-assisted computation offloading process as a QoE-aware utility maximization problem based on multidimensional indicators and propose a cache-assisted collaborative task offloading and resource allocation strategy to solve it.

III. System Model

In this section, we will specifically introduce the system model which is composed by four parts: the MEC enabled IoV network model, the request model, the energy harvesting model and the caching model. The specific execution processing of the system is denoted in Fig. 2 which denotes that the uplink or downlink transmission can via either the MBS or RSU, and a backhaul is existed between the RSU and the MBS.

A. The MEC Enabled IoV Network Model

As is denoted in Fig. 1, we study a MEC enabled IoV network which includes N vehicles, some RSUs, a MBS equipped with a MEC server deployed near a intersection, and the central cloud. In this system, the service area of the RSU is overlaid by the MBS, i.e., both the MBS and the RSUs operate in the same frequency band. The RSU is responsible for sending and receiving data for vehicles or the MBS, which can be regarded as a Small Base Station (SBS). The spectrum among the RSUs and the vehicles are divided into K identical subchannels, and the set of subchannels is denoted as $\mathcal{K} = \{1, 2 \cdots, k, \ldots, K\}$, where $|\mathcal{K}| = K$. The sub-channels are assumed to be Orthogonal Frequency Division Multiplexing (OFDM)-based and the bandwidth is identical and can be denoted as B. A backhaul between the RSU and the MBS is used to relay the transmission from the RSU to the MBS. The MEC server can execute the computation requests on behalf of the vehicles. In addition, the MEC server has the function of caching and the maximum cache space is $\mathcal{O}$, which can store some requests' database data and be contributed to the acceleration of task processing. The vehicle can offload the computation requests to the MEC server through either the MBS or the RSU, but only one mode can be chose. In this article, we also assume that the studied time is divided into discrete time slots, which are expressed as $\mathcal{T} = \{0, 1 \cdots, t \cdots, T-1\}$ and slot length is τ.

B. Request Model

In the MEC enabled IoV network, the set of vehicles is listed as $\mathcal{N} = \{1, 2, \ldots, N\}$, which are completely overlaid by the RSUs. For each vehicle, the full duplex communication interface is used to establish wireless communication with the nearest RSU.

For the sake of presentation, the term vehicle $i(t)$ means vehicle i at time slot t. Specially, the requests generated from vehicle $i(t), i \in \mathcal{N}$ is denoted as $RE_i(t)$ that is characterized as $RE_i(t) = \{\mathcal{C}_i(t), \mathcal{X}_i(t), \mathcal{G}_i(t)\}$, where $\mathcal{C}_i(t)$ denotes the number of CPU cycles required for processing unit request data, the request data consists of two parts,local dynamic collected data $\mathcal{X}_i(t)$ and the relevant database data $\mathcal{G}_i(t)$. Among $\mathcal{X}_i(t)$ only store in the vehicle, $\mathcal{G}_i(t)$ is essential to compute the request, which is only stored in the MEC server or the central cloud and belongs to the unchange data. Each request can be executed locally or offloaded to the MEC server through either the RSU or the MBS, but only one mode can be selected. Also we assume that each request is indivisible, we denote the $p_i(t) \in \{0, 1\}$ as the offloading strategy for vehicle $i(t)$, where $p_i(t) = 0$ denotes the request is executed locally, and $p_i(t) = 1$ denotes the request is offloaded to the MEC server.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

C. Energy Harvesting Model

In the future IoV, energy harvesting technology will be more widely used in electric vehicles. In this article we assume that the vehicles have equipped with the capacity of energy harvesting through continuously collecting energy packets from solar. We assume that the arrival of energy packets obeys Poisson process with an average rate $e_i(t)$, which satisfies the condition of $0 < e_i(t) \leq e_i^{\max}(t)$, with no doubt that $e_i^{\max}(t)$ denotes the maximum arrival rate of energy packets and it varies in different time slots. The collected energy packet will be stored in the vehicle's CPU battery, whose specific purpose is for local CPU execution, data receiving or requests offloading. Let $\hat{B}_i(t)$ denotes the quantity of energy packets at the beginning of time slot t for vehicle i. In addition, we have the condition that $\hat{B}_i(t) < \infty$, $\forall t \in T$.

D. Caching Model

In this article, we assume that the caching management is performed within each time interval, which is different from the general case that caching updating is occurred within a long-term-scale. MEC server would remove the relevant database data which are less used and retain and add those frequently used at the end of the time slot in order to improve system efficiency such as reduce the total execution delay and energy consumption and so on. In this way, vehicles can take full advantage of the caching resource, so as to adjust the vehicles' execution schemes in real time. Generally, due to lack adequate storage space for MEC server, the data may not be cached on MEC server overall. If the relevant database data are cached, the vehicles can download them from the MEC server directly through either the MBS or the RSU. Otherwise, vehicles can only download them from the cloud server through the MEC server, then with the assistance of the MBS or the RSU, the data can be transmitted to the vehicles. We assume that at the beginning of time slot t, the caching decisions can be expressed as $\phi_i(t) \in \{0, 1\}$, where $\phi_i(t) = 0$ means the revelent data of task $RE_i(t)$ is not cached on MEC, and $\phi_i(t) = 1$ shows that the relevant data is cached on MEC server. Assuming the maximum storage space is $\mathcal{O}$, and the data caching decision should satisfy the following constraint:

$$\sum_{i \in \mathcal{N}} \phi_i(t) \mathcal{G}_i(t) \leq \mathcal{O} \tag{1}$$

After the requests have completed execution in the MEC server, the server would update its caching strategies at the end of the time slot t. Let $\psi_i(t) \in \{0, 1, -1\}$ denote the caching updating strategies for request $RE_i(t)$, where $\psi_i(t) = 0$ means the cached status remains the same, $\psi_i(t) = 1$ shows that the data will be cached at the MEC server and $\psi_i(t) = -1$ indicates that the cached data in MEC server will be deleted. Furthermore, the caching update decisions satisfy the following constraint.

$$\psi_i(t) \geq -\phi_i(t) \tag{2}$$

It is worth noting that the meaning of (2) is that the MEC server cannot remove the no-cached requests.

TABLE I
SUMMARY OF THE KEY NOTATIONS

Notations	Meanings
$\mathcal{N}$	the set of vehicles in the system
N	the total number of vehicles in the system
$\mathcal{T}$	the set of time slots
T	the total number of time slots in the system
τ	the length of each time slot
$\mathcal{K}$	the set of subchannels
K	the total number of subchannels
$\mathcal{C}_i(t)$	the number of CPU cycles for processing unit request data
$\mathcal{X}_i(t)$	the size of the dynamic collected data
$\mathcal{G}_i(t)$	the size of revelent database data
$\phi_i(t)$	the cached status of vehicle i
$\psi_i(t)$	the caching updating strategy of vehicle i
$\mathcal{O}$	the maximum storage space of MEC server
$p_i(t)$	the offloading strategy of vehicle i
B	the sub-channel bandwidth
$p_{i,k}^{S,down}(t)$	the downlink transmission power through RSU
$h_{i,k}^{S,down}(t)$	the downlink channel gain through RSU
$d_{i,k}^{S,down}$	the distance between vehicle i and the closest RSU
$\rho_{i,j,k}^{down}(t)$	the revelent data downloading indicator
$p_{i,k}^{M,down}(t)$	the downlink transmission power through MBS
$h_{i,k}^{M,down}(t)$	the downlink channel gain through MBS
$p_{i,k}^{S,up}(t)$	the uplink transmission power through RSU
$h_{i,k}^{S,up}(t)$	the uplink channel gain through RSU
$d_{i,k}^{S,up}$	the distance between vehicle i and the closest RSU
$d_{i,k}^{M}(t)$	the distance between vehicle i and the MBS
$\rho_{i,j,k}^{up}(t)$	the request offloading indicator
$p_{i,k}^{M,up}(t)$	the uplink transmission power through MBS
$h_{i,k}^{M,up}(t)$	the uplink channel gain through MBS
σ^2	the background noise power
α	the path fading factor
R_{mc}	the downlink transmission rate from cloud server to MEC server
ζ	the scaling factor of backhaul
$f_i(t)$	the service rate of vehicle i
k_i	the switched capacitance of vehicle i
$f_M(t)$	the service rate of MEC server
k_M	the switched capacitance of MEC server
$e_i(t)$	the harvested energy for vehicle i
$e_i^{\max}(t)$	the maximum harvested energy for vehicle i
$\hat{B}_i(t)$	the total energy for vehicle i
ω_i	the weight factor of vehicle i

And the cached status is updated according by:

$$\phi_i(t+1) = \phi_i(t) + \psi_i(t); \sum_{i \in \mathcal{N}} (\phi_i(t) + \psi_i(t)) G_i(t) \leq \mathcal{O} \tag{3}$$

where $\phi_i(t+1) \in \{0, 1\}$ is the caching strategy for request RE_i at be beginning of time slot $t+1$. Also it should not exceed the maximum storage space after caching updating.

To improve the readability, Table I lists the key notations, $*(t)$ all refer to $*$ at time slot t.

IV. Request Execution Process

This section will discuss two cases where the requests are executed locally or offloaded to the MEC server through either the RSU or the MBS. Note that only one mode can be chose, no matter the execution mode or the transmission path selection mode.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

A. Local Execution Process

If the vehicle $i(t)$ executes the request $RE_i(t)$ locally, the revelent database data need to be downloaded from MEC server if it is cached or the cloud server. We assume that the RSU closest to the vehicle $i(t)$ or the MBS is responsible for the relevant data downloading transmission.

1) Downlink Transmission: In the case, the relevant database data is transmitted via the closest RSU, and the picked RSU select subchannel k to access vehicle $i(t)$, then obtained downlink data transmission rate $r_{i,k}^{S,down}(t)$ can be displayed as follows:

$$r_{i,k}^{S,down}(t) = B\log_2\left(1 + \frac{p_{i,k}^{S,down}(t)h_{i,k}^{S,down}(t)}{I_{i,k}^{M,down} + \sigma^2}\right) \quad (4)$$

where $p_{i,k}^{S,down}(t)$ is the downlink transmission power, $h_{i,k}^{S,down}(t)$ indicates the downlink channel gain between vehicle $i(t)$ and the worked RSU, σ^2 is the background noise power, and $I_{i,k}^{M,down}$ denotes the interference at the RSU on subchannel k, which is caused by other downlink transmission from the MBS to the vehicle on the same channel. Also, as the position of the vehicle is changing with time and the value of $h_{i,k}^{S,down}$ is associated with the distance between vehicle $i(t)$ and the closest RSU, so $h_{i,k}^{S,down}(t) = (d_{i,k}^{S,down}(t))^{\alpha}$, where $d_{i,k}^{S,down}$ denotes the distance between vehicle $i(t)$ and the closest RSU in the downlink on subchannel k and α means the path fading factor.

In addition, the worked RSU can select several subchannels to process downlink transmission simultaneously, then the total downlink transmission rate $R_i^{S,down}(t)$ through the worked RSU is denoted as follows:

$$\begin{aligned} R_i^{S,down}(t) &= \sum_{k=1}^{K} \rho_{i,0,k}^{down}(t) r_{i,k}^{S,down}(t) \\ &= \sum_{k=1}^{K} \rho_{i,0,k}^{down}(t) B\log_2\left(1 + \frac{p_{i,k}^{S,down}(t)h_{i,k}^{S,down}(t)}{I_{i,k}^{M,down} + \sigma^2}\right) \end{aligned} \quad (5)$$

Similarly, in the case, the transmission of the relevant database data is executed via the MBS, and an arbitrary subchannel such as subchannel k is picked up to access vehicle $i(t)$, the downlink data transmission rate $r_{i,k}^{M,down}(t)$ can be displayed as follows:

$$r_{i,k}^{M,down}(t) = B\log_2\left(1 + \frac{p_{i,k}^{M,down}(t)h_{i,k}^{M,down}(t)}{I_{i,k}^{S,down} + \sigma^2}\right) \quad (6)$$

where $p_{i,k}^{M,down}(t)$ is the downlink transmission power through the MBS; $h_{i,k}^{M,down}(t)$ is the downlink channel gain, which is related to the distance between the vehicle $i(t)$ and the MBS and can be denoted as $h_{i,k}^{S,down}(t) = (d_{i,k}^{S,down}(t))^{\alpha}$, where $d_{i,k}^{S,down}$ is the distance and α means the path fading factor; $I_{i,k}^{S,down}$ denotes the interference at the MBS on subchannel k, which is caused by other downlink transmission from the RSU to the vehicle $i(t)$ on the same channel; and σ^2 is the background noise power.

Also, the MBS can also pick up a number of subchannels to transmit the downlink data at the same time, then the total downlink transmission rate through the MBS is denoted as follows:

$$\begin{aligned} R_i^{M,down}(t) &= \sum_{k=1}^{K} \rho_{i,1,k}^{down}(t) r_{i,k}^{M,down}(t) \\ &= \sum_{k=1}^{K} \rho_{i,1,k}^{down}(t) B\log_2\left(1 + \frac{p_{i,k}^{M,down}(t)h_{i,k}^{M,down}(t)}{I_{i,k}^{S,down} + \sigma^2}\right) \end{aligned} \quad (7)$$

For the sake of presentation, we introduce a variable $\rho_{i,j,k}^{down}(t)$ called relevant database data downloading indicator for vehicle $i(t)$, which includes the downlink transmission path selection and subchannel assignment decision, where $\rho_{i,j,k}^{down}(t) = 1$ means the mode j through subchannel k is chose to accomplish the relevant data downloading for vehicle $i(t)$; $\rho_{i,j,k}^{down}(t) = 0$, otherwise. Here, $i \in \mathcal{N}$, $k \in \mathcal{K}$, and $j = \{0, 1\}$. $j = 0$ denotes that downlink transmission through the RSU, and $j = 1$ means that downlink transmission through the MBS.

In this way, the value of the interference $I_{i,k}^{M,down}$ can be denoted as $I_{i,k}^{M,down} = \sum_{l=1,l\neq i}^{N} \rho_{l,1,k} p_{l,k}^{M,down}(t) h_{l,k}^{S,down}(t)$, and the corresponding interference $I_{i,k}^{S,down}$ can be expressed as $I_{i,k}^{S,down} = \sum_{l=1,l\neq i}^{N} \rho_{l,0,k}^{down} p_{l,k}^{S,down}(t) h_{l,k}^{M,down}(t)$.

As the limited backhaul existed between the MBS and the RSU, we assume that the transmission delay of the backhaul is proportional to the length of data with the scaling factor ζ. So the backhaul delay between the RSU and vehicle $i(t)$ can be denoted as $D_{i,S}^{bh,down}(t) = \zeta \mathcal{G}_i$.

In summary, the total downlink transmission time through RSU to vehicle $i(t)$, which is denoted as $D_{i,S}^{down}(t)$, and is displayed as follows:

$$\begin{aligned} D_{i,S}^{down}(t) = D_{i,S}^{bh,down} &+ \phi_i(t)\frac{\mathcal{G}_i(t)}{R_i^{S,down}(t)} \\ &+ (1 - \phi_i(t))\left(\frac{\mathcal{G}_i(t)}{R_i^{S,down}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}}\right) \end{aligned} \quad (8)$$

In other words, if the revelent database data is cached, the corresponding downlink transmission time is $D_{i,S}^{bh,down} + \phi_i(t)\frac{\mathcal{G}_i(t)}{R_i^{S,down}(t)}$, otherwise, the corresponding downlink transmission time is $D_{i,S}^{bh,down} + (1 - \phi_i(t))(\frac{\mathcal{G}_i(t)}{R_i^{S,down}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}})$, where R_{mc} denotes the downlink transmission rate from central cloud server to the MEC server.

On the other hand, the total downlink transmission time $D_{i,M}^{down}(t)$ from MBS to vehicle i is given as:

$$\begin{aligned} D_{i,M}^{down}(t) = \phi_i(t)&\frac{\mathcal{G}_i(t)}{R_i^{M,down}(t)} \\ &+ (1 - \phi_i(t))\left(\frac{\mathcal{G}_i(t)}{R_i^{M,down}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}}\right) \end{aligned} \quad (9)$$

where the expression of $\phi_i(t)\frac{\mathcal{G}_i(t)}{R_i^{S,down}(t)}$ denotes the downlink transmission time of the revelent data of $RE_i(t)$ through MBS under the case that the MEC server cached the data, while $(1 -$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

$\phi_i(t))(\frac{\mathcal{G}_i(t)}{R_i^{S,down}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}})$ indicates the case that the revelent data is not cached.

To sum up, whether through MBS or RSU, the total downlink transmission time $D_i^{down}(t)$ can be displayed as

$$D_i^{down}(t) = \max\left\{\rho_{i,0,k}^{down}(t), k \in K\right\} D_{i,S}^{down}(t) + \max\left\{\rho_{i,1,k}^{down}(t), k \in K\right\} D_{i,M}^{down}(t) \quad (10)$$

For the above formula, $\max\{*\}$ is a judging condition, such as: if $\max\{\rho_{i,0,k}^{down}(t), k \in K\} = 1$, while $\max\{\rho_{i,1,k}^{down}(t), k \in K\} = 0$, then the RSU is selected as the transmission path, and the according total downlink transmission time $D_i^{down}(t) = D_{i,S}^{down}(t)$; and vice versa.

Then the total energy consumption $E_i^{down}(t)$ of receiving the downlink data can be given as follows:

$$E_i^{down}(t) = \sum_{k=1}^{K} \rho_{i,1,k}^{down}(t) p_{i,k}^{M,down}(t) D_{i,M}^{down}(t) + \sum_{k=1}^{K} \rho_{i,0,k}^{down}(t) p_{i,k}^{S,down}(t) D_{i,S}^{down}(t) \quad (11)$$

Among, the expression of $\sum_{k=1}^{K} \rho_{i,1,k}^{down}(t) p_{i,k}^{M,down}(t) D_{i,M}^{down}(t)$ is the energy consumption of receiving the downlink data through the MBS; while $\sum_{k=1}^{K} \rho_{i,0,k}^{down}(t) p_{i,k}^{S,down}(t) D_{i,S}^{down}(t)$ is the energy consumption of receiving the downlink data through the RSU.

2) Local Execution Delay and Energy Consumption: When the relevant database data is arrived at the vehicle $i(t)$, the vehicle i can execute the request locally based on its CPU server, and the total local execution delay $D_i^L(t)$ can be given as

$$D_i^L(t) = \frac{\mathcal{V}_i(t)\mathcal{C}_i(t)}{f_i} \quad (12)$$

where f_i is the service rate of vehicle i.

It is worth mentioning that the power consumption of CPU processor is proportion to the 3-power of the CPU-cycle frequency in the situation of a low CPU voltage. So the corresponding energy consumption $E_i^L(t)$ for local execution of vehicle i can be denoted as

$$E_i^L(t) = \kappa_i f_i^3 D_i^L(t) \quad (13)$$

where κ_i is a constant which is determined by the switched capacitance of vehicle i and can be obtained from its configuration parameters.

B. Request Offloading Process

With the target to reduce the energy consumption or execution delay, or something else, the vehicles can also offload their requests to the MEC server through either the MBS or the RSU. Firstly, for clarity, we define $\rho_{i,j,k}^{up}(t)$ as the request offloading indicator of vehicle $i(t)$, which includes the uplink transmission path selection and subchannel assignment decision, where $\rho_{i,j,k}^{up}(t) = 1$ means vehicle $i(t)$ choose mode j through subchannel k to accomplish the requests offloading, $\rho_{i,j,k}^{up}(t) = 0$, otherwise. Here, $i \in \mathcal{N}$, $k \in \mathcal{K}$, and $j = \{0, 1\}$. $j = 0$ denotes that uplink transmission through the RSU, $j = 1$ means that uplink transmission through the MBS.

1) Uplink Transmission: If the MBS is selected by vehicle $i(t)$ on subchannel k to offload its requests, the obtained uplink data transmission rate $r_{i,k}^{M,up}(t)$ can be shown as:

$$r_{i,k}^{M,up}(t) = B\log_2\left(1 + \frac{p_{i,k}^{M,up}(t) h_{i,k}^{M,up}(t)}{I_{i,k}^{S,up} + \sigma^2}\right) \quad (14)$$

where $p_{i,k}^{M,up}(t)$ denotes the transmission power of vehicle $i(t)$ through MBS on the k-th subchannel; $h_{i,k}^{M,up}(t)$ indicates the channel gain and the the value of $h_{i,k}^{M,up}(t)$ is related to the distance between the vehicle $i(t)$ and the MBS, which is denoted as $h_{i,k}^{M,up}(t) = (d_{i,k}^{M,up}(t))^{\alpha}$; $I_{i,k}^{S,up}$ denotes the interference at the MBS on sub-channel k, which is caused by other vehicles' uplink transmission to the RSU on the same channel, and σ^2 is the background noise power.

Similar to the downlink transmission, vehicle $i(t)$ can pick up several subchannels to offload the dynamic collected data simultaneously, then the total uplink transmission rate $R_i^{M,up}(t)$ can be denoted as follows:

$$R_i^{M,up}(t) = \sum_{k=1}^{K} \rho_{i,1,k}^{up}(t) r_{i,k}^{M,up}(t) = \sum_{k=1}^{K} \rho_{i,1,k}^{up}(t) B\log_2\left(1 + \frac{p_{i,k}^{M,up}(t) h_{i,k}^{M,up}(t)}{I_{i,k}^{S,up} + \sigma^2}\right) \quad (15)$$

where $I_{i,k}^{S} = \sum_{l=1,l\neq i}^{N} \rho_{l,0,k}^{up} p_{l,k}^{S,up}(t) h_{l,k}^{M}(t)$, so $R_i^{M,up}(t)$ can be expressed as

$$R_i^{M,up}(t) = \sum_{k=1}^{K} \rho_{i,1,k}^{up}(t) \times B\log_2\left(1 + \frac{p_{i,k}^{M,up}(t) h_{i,k}^{M,up}(t)}{\sum_{l=1,l\neq i}^{N} \rho_{l,0,k}^{up} p_{l,k}^{S,up}(t) h_{l,k}^{M,up}(t) + \sigma^2}\right) \quad (16)$$

We use $D_{i,M}^{up}(t)$ as the uplink transmission delay for vehicle $i(t)$ through the MBS, so the $D_{i,M}^{up}(t)$ can be denoted as follows:

$$D_{i,M}^{up}(t) = \phi_i(t)\frac{\mathcal{X}_i(t)}{R_i^{M,up}(t)} + (1 - \phi_i(t))\left(\frac{\mathcal{X}_i(t)}{R_i^{M,up}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}}\right) \quad (17)$$

where $\phi_i(t)\frac{\mathcal{X}_i(t)}{R_i^{M,up}(t)}$ denotes the transmission time of offloading the dynamic data through the MBS if the revelent data is cached; while $(1 - \phi_i(t))(\frac{\mathcal{X}_i(t)}{R_i^{M,up}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}})$ indicates the transmission delay includes both the dynamic data offloading and revelent data downloading through the MBS when the revelent data is not cached.

If the nearest RSU is selected by vehicle $i(t)$ through accessing subchannel k, the reachable uplink data transmission rate $r_{i,k}^{S,up}(t)$ can be displayed as follows:

$$r_{i,k}^{S,up}(t) = B\log_2\left(1 + \frac{p_{i,k}^{S,up}(t) h_{i,k}^{S,up}(t)}{I_{i,k}^{M,up} + \sigma^2}\right) \quad (18)$$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

where $p_{i,k}^{S,up}(t)$ denotes the transmission power of vehicle $i(t)$; $h_{i,k}^{S,up}(t)$ indicates the channel gain between the vehicle $i(t)$ and the RSU, and satisfying $h_{i,k}^{S,up}(t) = (d_{i,k}^{S,up}(t))^{\alpha}$; $I_{i,k}^{M,up}$ denotes the interference at the RSU on subchannel k, which is caused by other vehicles' uplink transmission to the MBS on the same channel.

Then, the total uplink transmission rate $R_i^{S,up}(t)$ via selecting multiple channels at the same time can be denoted as:

$$R_i^{S,up}(t) = \sum_{k=1}^{K} \rho_{i,0,k}^{up}(t) r_{i,k}^{S,up}(t) = \sum_{k=1}^{K} \rho_{i,0,k}^{up}(t) B \log_2 \left(1 + \frac{p_{i,k}^{S,up}(t) h_{i,k}^{S,up}(t)}{I_{i,k}^{M,up} + \sigma^2}\right) \tag{19}$$

where $I_{i,k}^{M,up} = \sum_{l=1,l\neq i}^{N} \rho_{l,1,k}^{up} p_{l,k}^{M,up}(t) h_{l,k}^{S,up}(t)$, so $R_i^{S,up}(t)$ can be rewritten as:

$$R_i^{S,up}(t) = \sum_{k=1}^{K} \rho_{i,0,k}^{up}(t) B \log_2 \left(1 + \frac{p_{i,k}^{S,up}(t) h_{i,k}^{S,up}(t)}{\sum_{l=1,l\neq i}^{N} \rho_{l,1,k} p_{l,k}^{M,up}(t) h_{l,k}^{S,up}(t) + \sigma^2}\right) \tag{20}$$

Similarly, because of the limited backhaul between the RSU and the MBS, the backhaul delay $D_{i,S}^{bh,up}(t)$ can be denoted as $D_{i,S}^{bh,up}(t) = \zeta \mathcal{X}_i$.

In summary, the total uplink transmission time of vehicle $i(t)$ through RSU, which is denoted as $D_{i,S}^{up}(t)$, and is displayed as follows:

$$D_{i,S}^{up}(t) = D_{i,S}^{bh,up} + \phi_i(t) \frac{\mathcal{X}_i(t)}{R_i^{S,up}(t)} + (1 - \phi_i(t)) \times \left(\frac{\mathcal{X}_i(t)}{R_i^{S,up}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}}\right) \tag{21}$$

Among, the expression of $D_{i,S}^{bh,up}$ denotes the backhual delay between the RSU and the MBS, the epression of $\phi_i(t) \frac{\mathcal{X}_i(t)}{R_i^{S,up}(t)}$ indicates the uplink transmission of dynamic data under the situation of the revelent data is cached; the epression of $(1 - \phi_i(t))(\frac{\mathcal{X}_i(t)}{R_i^{S,up}(t)} + \frac{\mathcal{G}_i(t)}{R_{mc}})$ includes both the dynamic data offloading and revelent data downloading.

To sum up, no matter which mode is selected by vehicle $i(t)$, the uplink transmission time $D_i^{up}(t)$ can be displayed as

$$D_i^{up}(t) = \max\left\{\rho_{i,0,k}^{up}(t), k \in K\right\} D_{i,S}^{up}(t) + \max\left\{\rho_{i,1,k}^{up}(t), k \in K\right\} D_{i,M}^{up}(t) \tag{22}$$

If the RSU is selected as the transmission path for offloading, then $\max\{\rho_{i,1,k}^{up}(t), k \in K\} = 1$, while $\max\{\rho_{i,0,k}^{up}(t), k \in K\} = 0$, then the total uplink transmission time $D_i^{up}(t) = D_{i,M}^{up}(t)$; and vice versa.

Then the uplink transmission energy consumption $E_i^{up}(t)$ can be expressed as follows:

$$E_i^{up}(t) = \sum_{k=1}^{K} \rho_{i,1,k}^{up}(t) p_{i,k}^{M,up}(t) D_{i,M}^{up}(t) + \sum_{k=1}^{K} \rho_{i,0,k}^{up}(t) p_{i,k}^{S,up}(t) D_{i,S}^{up}(t) \tag{23}$$

where the expression of $\sum_{k=1}^{K} \rho_{i,1,k}^{up}(t) p_{i,k}^{M,up}(t) D_{i,M}^{up}(t)$ denotes the uplink transmission energy consumption if offloading through the MBS; while $\sum_{k=1}^{K} \rho_{i,0,k}^{up}(t) p_{i,k}^{S,up}(t) D_{i,S}^{up}(t)$denotes the uplink transmission energy consumption if offloading through the SBS, which are according to the selecting of transmission path.

2) MEC Execution Delay: When the requests from different vehicles are pooled to the MEC server, the MEC server can process them on behalf of the vehicles with its relatively sufficient computing resources. We assume that the service rate of MEC server is f_M, which is the CPU cycles it can process per unit time and far greater than the vehicle's execution ability. Then the MEC execution delay is given as

$$D_i^M(t) = \frac{\sum_{i=1}^{N} p_i(t) \mathcal{V}_i(t) \mathcal{C}_i(t)}{f_M} \tag{24}$$

The data size of the generated result is usually very small after the execution is completed on the MEC server, so we justly ignore the delay and energy consumption of those result returning for simplicity.

V. Problem Formulation

As is known to all, the traffic flow in IoV is obviously differentiated and asymmetric for peak hours (such as 7:30-9:30 am, 17:00–19:00 pm) and off-peak hours (for example, the midnight period). During the peak hours, the number of vehicles accessing the IoV will increase dramatically, resulting in the shortage of wireless subchannel resources and MEC computation resources, but in the off-peak hours, the amount of vehicles accessing the IoV is relatively few, and the wireless subchannel resources and MEC computation resources is adequate. In order to improve the comprehensive utilization rate of wireless and computation resources and the system performance, it is necessary to construct a reasonable mathematical model to satisfy the dynamic resource allocation. Thus, we divided the problem in two situations: the off-peak hours and the peak hours.

A. Problem Formulation for Off-Peak Hours

In this case, as the subchannels or the MEC computation resource are relatively sufficient, the vehicles can directly offload their requests to the MEC servers through subchannels' transmission instead of execution locally. Under this premise, we can reduce the total execution delay and energy consumption based on the constructed system model.

1) Total Execution Delay: According to the derive MEC execution process in the previous section, as the vehicles can directly offload their requests to the MEC servers through subchannels'

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

transmission instead of execution locally, so the execution delay includes the uplink transmission time $D_i^{up}(t)$ and the MEC execution delay $D_i^M(t)$. We can derived the total execution delay for off-peak hours as follows:

$$D_i^{idle}(t) = \left(D_i^{up}(t) + D_i^M(t)\right) = \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{up}(t), k \in K\right\} D_{i,S}^{up}(t) \\ +\max\left\{\rho_{i,1,k}^{up}(t), k \in K\right\} D_{i,M}^{up}(t) + D_i^M(t) \end{array} \right\} \quad (25)$$

On the basis of (25), we give each vehicle a weight, different vehicles in the system have different proportion coefficients as similar with social network, which is also adopted by many literature, then the average weighted execution delay of the studied system for all the participating vehicles at time slot t is denoted as $WD_{total}^{idle}(t)$, which is given as

$$WD_{total}^{idle}(t) = \sum_{i=1}^{N} \frac{1}{N}\omega_i \cdot \left\{ \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{up}(t), k \in K\right\} D_{i,S}^{up}(t) + \\ \max\left\{\rho_{i,1,k}^{up}(t), k \in K\right\} D_{i,M}^{up}(t) + D_i^M(t) \end{array} \right\} \right\} \quad (26)$$

where ω_i is the weight factor of vehicle i, and this parameter mirror the relative importance in the studied set.

Then on the basis of (26), which denotes the average weighted execution delay of the studied system for all the participating vehicles at time slot t. Then we derive the expressions of the average execution delay $MWD_{total}^{idle}(t)$ of the MEC enabled IoV system during T time slots for off-peak hours, which is given as follows:

$$MWD_{total}^{idle}(t) = \lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} WD_{total}^{idle}(t) = \lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i=1}^{N} \frac{1}{N}\omega_i \cdot \left\{ \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{up}(t), k \in K\right\} D_{i,S}^{up}(t) + \\ \max\left\{\rho_{i,1,k}^{up}(t), k \in K\right\} D_{i,M}^{up}(t) + D^M{}_i(t) \end{array} \right\} \right\} \quad (27)$$

In the situation of off-peak hours, we aim to minimize $MWD_{total}^{idle}(t)$ as far as possible. We denote the system's total decision at time slot t as $\boldsymbol{V}(t) = [\boldsymbol{\Phi}(t), \boldsymbol{\Psi}(t), \boldsymbol{\rho^{up}}(t)]$, $\forall t \in \mathcal{T}$, where $\boldsymbol{\Phi}(t) = [\phi_1(t), \phi_2(t), \ldots, \phi_i(t), \ldots, \phi_N(t)]$ is the caching decision at the beginning of the time slot t, $\boldsymbol{\Psi}(t) = [\psi_1(t), \psi_2(t), \ldots, \psi_i(t), \ldots, \psi_N(t)]$ is the caching updating strategy at the end of the time slot t, $\boldsymbol{\rho}^{up}(t) = [\rho_{1,j,k}^{up}(t), \rho_{2,j,k}^{up}(t), \ldots, \rho_{N,j,k}^{up}(t)]$ is the decisions of transmission path selection and subchannel assignment for all vehicles at time slot t.

In the situation of off-peak hours, for simplicity, energy consumed for other purposes aparting from requests transmission is neglect. The energy consumed by vehicle $i(t)$ is shown as $E_{i,total}^{idle}(t)$, which only comprises the energy consumption for sending requests, expressed as $E_i^{up}(t)$, and which depends on the strategy it selected. We can show the expression of $E_{i,total}^{idle}(t)$ as follows:

$$E_{i,total}^{idle}(t) = E_i^{up}(t) \quad (28)$$

Note that $E_{i,total}^{idle}(t)$ cannot exceed the current available battery level, i.e.,

$$E_{i,total}^{idle}(t) \le \hat{B}_i(t) \quad (29)$$

The harvested energy at time slot t will be used for the next time slot $t+1$. Thus, the evolution of the battery energy level of vehicle i obeys the following equation:

$$\hat{B}_i(t+1) = \hat{B}_i(t) - E_{i,total}^{idle}(t) + e_i(t) \quad (30)$$

2) The Optimization Formulation: Therefore, the problem for the situation of the off-peak hours is formulated in the following:

$$\mathbf{P1}: \min_{\boldsymbol{V}(t)} MWD_{total}^{idle}(t) \quad (31)$$

s.t.

$$\phi_i(t) \in \{0,1\} \quad (32a)$$

$$\psi_i(t) \in \{0,1,-1\} \quad (32b)$$

$$\rho_{i,j,k}^{up}(t) \in \{0,1\} \quad (32c)$$

$$\sum_{j=0}^{1} \rho_{i,j,k}^{up}(t) = 1 \quad (32d)$$

$$E_{i,total}^{idle} \le \hat{B}_i(t) \quad (32e)$$

$$\psi_i(t) \ge -\phi_i(t) \quad (32f)$$

$$\sum_{i\in\mathcal{N}} \phi_i(t)\mathcal{G}_i(t) \le \mathcal{O} \quad (32g)$$

$$\sum_{i\in\mathcal{N}} \left(\phi_i(t) + \psi_i(t)\right)\mathcal{G}_i(t) \le \mathcal{O} \quad (32h)$$

$$i \in \mathcal{N}, t \in \mathcal{T}, k \in \mathcal{K} \quad (32i)$$

Nevertheless, due to the energy causality constraints (30), the vehicles' decisions are constrained by successive time slots. It makes the problem have degree of difficulty to be tackled. Thus by introducing a non-negative value $E_i^{\min}(t)$ as the lower bound, and a reasonable value of $E_i^{\max}(t)$ as upper bound iteratively, elimination of the related coupling condition can be finished without considering the constraints (30). Thus, we rewrite an improved version of the above problem as follows:

$$\mathbf{P1}: \min_{\boldsymbol{V}(t)} MWD_{total}^{idle}(t)$$

Subject to

$$(37a),\ (37b),\ (37c),\ (37d),\ (37f),\ (37g),\ (37h) \quad (33)$$

$$E_{i,total}^{idle}(t) \in \left[E_i^{\min}(t), E_i^{\max}(t)\right] \quad (34)$$

With the idea of estimation, the (32e) can be transformed into (34), the energy constraint can be used during the process of optimization. For simplification, when $E_i^{\min}(t)$ approaches to 0,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

the optimal solution of P1 and the original problem are identical. As $\mathbf{P1}$ is a stochastic optimization problem, there are a great deal of optimization variables needed to be decided, such as the caching strategy, the caching updating decision, and the uplink transmission path and subchannel assignment decision. In general, we can achieve the total optimal system decisions by a continual process of addressing the deterministic per time slot problem.

B. Problem Formulation for the Peak Hours

Next, we will discuss the situation of the peak hours. In this case, as more and more vehicles in the IoV compete the limited wireless and computation resource, so the requests generated from the vehicles can execute locally or be offloaded to the MEC servers through the subchannels.

1) Total Delay: According to the derived local execution and MEC execution process in the previous section, we can derive the total execution delay for the peak hours as follows:

$$\begin{aligned} D_i^{busy}(t) &= (1-p_i(t))(D_i^{down}(t)+D_i^L(t)) \\ &\quad + p_i(t)\left(D_i^{up}(t)+D_i^M(t)\right) \\ &= (1-p_i(t)) \\ &\times \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{down}(t), k\in K\right\} D_{i,S}^{down}(t) \\ +\max\left\{\rho_{i,1,k}^{down}(t), k\in K\right\} D_{i,M}^{down}(t)+D_i^L(t) \end{array} \right\} \\ &+ p_i(t) \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{up}(t), k\in K\right\} D_{i,S}^{up}(t) \\ +\max\left\{\rho_{i,1,k}^{up}(t), k\in K\right\} D_{i,M}^{up}(t)+D_i^M(t) \end{array} \right\} \end{aligned} \tag{35}$$

The total weighted execution delay of the system at time slot t is denoted as $WD_{total}^{busy}(t)$, which is given as

$$\begin{aligned} WD_{total}^{busy}(t) &= \sum_{i=1}^{N} \frac{1}{N}\omega_i \\ &\times \left\{ \begin{array}{l} (1-p_i(t)) \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{down}(t), k\in K\right\} D_{i,S}^{down}(t) \\ +\max\left\{\rho_{i,1,k}^{down}(t), k\in K\right\} D_{i,M}^{down}(t) \\ +D_i^L(t) \end{array} \right\} \\ +p_i(t) \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{up}(t), k\in K\right\} D_{i,S}^{up}(t) \\ +\max\left\{\rho_{i,1,k}^{up}(t), k\in K\right\} D_{i,M}^{up}(t)+D_i^M(t) \end{array} \right\} \end{array} \right\} \end{aligned} \tag{36}$$

Then we derive the average execution delay $MWD_{total}^{busy}(t)$ of the MEC computing system during T time slots, which is given as follows:

$$\begin{aligned} MWD_{total}^{busy}(t) &= \lim_{T\to\infty}\frac{1}{T}\sum_{t=0}^{T-1} WD_{total}^{busy}(t) \\ &= \lim_{T\to\infty}\frac{1}{T}\sum_{t=0}^{T-1}\sum_{i=1}^{N}\frac{1}{N}\omega_i \cdot \\ &\left\{ \begin{array}{l} (1-p_i(t)) \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{down}(t), k\in K\right\} D_{i,S}^{down}(t)+ \\ \max\left\{\rho_{i,1,k}^{down}(t), k\in K\right\} D_{i,M}^{down}(t)+ \\ D_i^L(t) \end{array} \right\} \\ +p_i(t) \left\{ \begin{array}{c} \max\left\{\rho_{i,0,k}^{up}(t), k\in K\right\} D_{i,S}^{up}(t)+ \\ \max\left\{\rho_{i,1,k}^{up}(t), k\in K\right\} D_{i,M}^{up}(t)+D_i^M(t) \end{array} \right\} \end{array} \right\} \end{aligned} \tag{37}$$

In peak hours, our aim is to minimize $MWD_{total}^{busy}(t)$ as far as possible. We denote the system's total decision at time slot t as $\boldsymbol{V}(t)=[\boldsymbol{p}(t),\boldsymbol{\Phi}(t),\boldsymbol{\Psi}(t),\boldsymbol{\rho}^{up}(t),\boldsymbol{\rho}^{down}(t)]$, $\forall t\in\mathcal{T}$, where $\boldsymbol{p}(t)=[p_1(t),p_2(t),\ldots,p_i(t),\ldots,p_N(t)]$ is the offloading strategies for all vehicles in the system at the time slot t, $\boldsymbol{\Phi}(t)=[\phi_1(t),\phi_2(t),\ldots,\phi_i(t),\ldots,\phi_N(t)]$ is the caching decisions at the beginning of the time slot t, $\boldsymbol{\Psi}(t)=[\psi_1(t),\psi_2(t),\ldots,\psi_i(t),\ldots,\psi_N(t)]$ is the caching updating strategy at the end of the time slot t, $\boldsymbol{\rho}^{up}(t)=[\rho_{1,j,k}^{up}(t),\rho_{2,j,k}^{up}(t),\ldots,\rho_{N,j,k}^{up}(t)]$ is the decisions of uplink transmission path selection and subchannel assignment for all vehicles at time slot t, and $\boldsymbol{\rho}^{down}(t)=[\rho_{1,j,k}^{down}(t),\rho_{2,j,k}^{down}(t),\ldots,\rho_{N,j,k}^{down}(t)]$ is the decisions of downlink transmission path selection and subchannel assignment for all vehicles at time slot t.

Similarly, we denote the energy consumed by the vehicle $i(t)$ as $E_{i,total}^{busy}(t)$, which comprises of three parts: 1) energy consumption of execution locally for vehicle $i(t)$, denoted as $E_i^L(t)$; 2) energy consumption for offloading the requests, expressed as $E_i^{up}(t)$; 3) energy consumption for receiving the relevant data, expressed as $E_i^{down}(t)$. As we can see that the energy consumption of those parts are depending on the strategy it chooses. We can express $E_{i,total}^{busy}(t)$ as follows:

$$\begin{aligned} E_{i,total}^{busy}(t) &= (1-p_i(t))(E_i^L(t)+E_i^{down}(t)) \\ &\quad + p_i(t)(E_i^{up}(t)) \end{aligned} \tag{38}$$

Note that $E_{i,total}^{busy}(t)$ cannot exceed the battery level, i.e.,

$$E_{i,total}^{busy}(t) \le \hat{B}_i(t) \tag{39}$$

And the battery energy level of vehicle $i(t)$ evolves according to the following equation:

$$\hat{B}_i(t+1) = \hat{B}_i(t) - E_{i,total}^{busy}(t) + e_i(t) \tag{40}$$

2) Optimization Formulation: Therefore, the average execution delay minimization problem for the peak hours can be formulated in the following

$$\mathbf{P2}: \min_{\boldsymbol{V}(t)} MWD_{total}^{busy}(t) \tag{41}$$

s.t.

$$p_i(t)\in\{0,1\} \tag{42a}$$

$$\phi_i(t)\in\{0,1\} \tag{42b}$$

$$\psi_i(t)\in\{0,1,-1\} \tag{42c}$$

$$\rho_{i,j,k}(t)\in\{0,1\} \tag{42d}$$

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

$$\sum_{j=0}^{1} \rho_{i,j,k}(t) = 1 \tag{42e}$$

$$E_{i,total}^{busy} \leq \hat{B}_i(t) \tag{42f}$$

$$\psi_i(t) \geq -\phi_i(t) \tag{42g}$$

$$\sum_{i \in \mathcal{N}} \phi_i(t) \mathcal{G}_i(t) \leq \mathcal{O} \tag{42h}$$

$$i \in \mathcal{N}, t \in \mathcal{T}, k \in \mathcal{K} \tag{42i}$$

Similarly, we introduce a modified version of the above problem with the idea of estimations as follows:

$$\mathbf{P2}: \min_{\boldsymbol{V}(t)} \ MWD_{total}^{busy}(t)$$

Subject to

$$(47a), (47b), (47c), (47\tilde{}d), (47e), (47\tilde{}g), (47\tilde{}h), (47i) \tag{43}$$

$$E_{i,total}^{busy}(t) \in \left[E_i^{\min}(t), E_i^{\max}(t)\right] \tag{44}$$

In summary, the caching strategy and the transmission path selection is same for the two situations, also we adopt the weight method to derive the average execution delay. But the execution strategy is different, so the construction forms of the total execution for vehicle i at time slot t is different for peak hours and off-peak hours. In addition,caching decision $\phi_i(t)$, caching updating decision $\psi_i(t)$, uplink transmission path selection and subchannel assignment strategy $\rho_{i,j,k}^{up}(t)$, $p_i(t)$ are all parameters that used to improve the QoS.

VI. Algorithm Design

For the two problems we constructed above, we design the Simulated Annealing Genetic Algorithm (SAGA) and DQN algorithm, which can switch freely between the off-peak hours and the peak hours.

A. Algorithm Design for the Off-Peak Hours

In the situation of off-peak hours, under the condition that the battery level is temporally related the optimization decision sets in different time slots are not independent. Therefore, the weighted perturbation method is introduced to solve this trouble. We define the perturbation parameter as η, and a virtual energy queue $B_i(t) = \hat{B}_i(t) - \eta_i$, where η_i is a bounded constant satisfying $\eta_i \geq E_{\max} + v\alpha_i / E_{\min}$. Then the Lyapunov optimization algorithm is firstly adopted to transform the original problem as multiple subproblems which are in the same time slot, and the detail process is shown below:

Step 1: Define the Lyapunov function:

$$L\left(\boldsymbol{B}(t)\right) = \frac{1}{2} \sum_{i \in \mathcal{N}} {B_i}^2(t) \tag{45}$$

where $\boldsymbol{B}(t) = [B_1(t), \ldots, B_i(t), \cdots B_N(t)]$.

Step 2: Define the the corresponding conditional Lyapunov drift:

$$\Delta\left(\boldsymbol{B}(t)\right) = \mathbb{E}\left[L\left(\boldsymbol{B}\left(t+1\right)\right) - L\left(\boldsymbol{B}(t)\right) | \boldsymbol{B}(t)\right] \tag{46}$$

Step 3: The Lyapunov drift-plus-penalty function:

$$\Delta_V\left(\boldsymbol{B}(t)\right) = \Delta\left(\boldsymbol{B}(t)\right) + V\mathbb{E}\left[WD_{total}^{idle}(t) | \boldsymbol{B}(t)\right] \tag{47}$$

where $V \in (0{+}\infty)$ is a control parameter in the proposed algorithm.

Step 4: Find an upper bound of $\Delta(\boldsymbol{B}(t))$ under any feasible set of $\boldsymbol{V}(t)$, and the upper bound can be expressed as

$$\begin{aligned}\Delta_V\left(\boldsymbol{B}(t)\right) \leq CT + \sum_{i \in \mathcal{N}} \left\{B_i(t)\left[e_i(t) - E_{i,total}^{idle}(t)\right]\right\} \\ + V\mathbb{E}\left[WD_{total}^{idle}(t) | \boldsymbol{B}(t)\right]\end{aligned} \tag{48}$$

where CT is a constant, which is denoted as

$$CT = \sum_{i \in N} \left[\frac{\left(e_i^{\max}(t)\right)^2 + \left(E_i^{\max}(t)\right)^2}{2}\right]$$

Step 5: Minimize the upper bound of $\Delta_V(\boldsymbol{B}(t))$ in the right-hand side of the above equation at each time slot t.

In other words, the problem **P1** can be transformed into solving the following problem:

$$\begin{aligned}\mathbf{P3}: \min_{\boldsymbol{V}(t)} \sum_{i \in \mathcal{N}} \left\{B_i(t)\left[e_i(t) - E_{i,total}^{idle}(t)\right]\right\} \\ + V\mathbb{E}\left[WD_{total}^{idle}(t) | \boldsymbol{B}(t)\right]\end{aligned}$$

s.t. (38) and (39)

In order to solve the Problem **P3**, we intend to adopt Simulated Annealing Genetic Algorithm (SAGA) [25], which is an perfect combination of genetic algorithm and simulated annealing algorithm [39], [40]. Usually, Genetic algorithm is strong in grasping the overall search process but poor in local search. On the other hand, simulated annealing algorithm does not good at the whole search space and the corresponding efficiency is not high, but has strong local search ability, which can avoid falling into local optimum in the search process. Through combining the two algorithms together, we can obtain the property of local search and global convergence.

The steps of SAGA contains:

Step 1: Initial annealing rate ϖ; temperature T_0; terminated temperature T_{final}; the number of inner cycles L for each T, $2r$ initial solutions of the problem $\mathbf{P3}$, (i.e., using the random method to find $2r$ solutions of $\mathbf{P3}$, which satisfies the conditions (38)–(39), and treat as the initial group, and obtain the fitness of the group, that is the value of the objective function of $\mathbf{P3}$); mutation operation γ make $T = T_0$; crossover operation $\hbar$; and a random number σ between [0,1].

Step 2: If $T > T_{final}$, we run the following steps 3, 4, 5; otherwise, return the optimal solution of the smallest fitness value;

Step 3: For each T and $i = 0, 1, \ldots, L$, run the following steps 4, 5; otherwise, make $T = \varpi T$ and return Step 2;

Step 4: (i) we randomly combine r pairs male parents among the group which includes $2r$ individuals;

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

(ii) for each pair of male parents $P_m(m = 1, 2, \ldots, r)$, including $P_m^{(1)}$ and , $P_m^{(2)}$, do the following two steps (iii)(iv)

(iii) generate the offspring $Q_m^{(1)}$ and $Q_m^{(2)}$ by using crossover operation $\hbar$ and mutation operation γ, and calculate the fitness of $Q_m^{(1)}$, $Q_m^{(2)}$, which is denoted as $f_{Q_m^{(1)}}$ $f_{Q_m^{(2)}}$.

(iv) If $f_{P_m^{(w)}} > f_{Q_m^{(w)}}$ $w = 1, 2$, replace $P_m^{(w)}$ by using $Q_m^{(w)}$; otherwise, calculate the value of $\exp(\frac{f_{Q_m^{(w)}} - f_{P_m^{(w)}}}{T})$, if $\sigma < \exp(\frac{f_{Q_m^{(w)}} - f_{P_m^{(w)}}}{T})$, then accept $Q_m^{(w)}$; otherwise, remain $P_m^{(w)}$.

(v) pool together the picked results.

Step 5: Return to Step 3.

B. Algorithm Design for the Peak Hours

In peak hours, involving the choice of offloading decision, caching decision, caching updating decision, transmission path selection and the allocation of channel resources $(p_i(t), \phi_i(t), \psi_i(t), \rho_{i,0,k}^{up}(t), \rho_{i,1,k}^{up}(t), \rho_{i,0,k}^{down}(t), \rho_{i,1,k}^{down}(t))$. Obviously, there are more variables when IoV is in the peak hours. The complexity of environment add more difficulty of solving the problem. In this subsection, we make use of algorithm for transmission path decision, subchannel assignment, offloading decision, caching status decision and caching update decision [41], [42], [43].

In our article, there are three elements in DQN: State $\boldsymbol{S}$, Action $\boldsymbol{A}$ and Reward R.

- State $\boldsymbol{S}$: The vehicular state includes three parts $\boldsymbol{S}(t) = (\boldsymbol{Pos}(t), \mathcal{O}(t), \boldsymbol{B}(t))$ at time slot t: $\boldsymbol{Pos}(t)$ means the position of vehicles, which can impact the quality of wireless communication; $\mathcal{O}$ means caching space of MEC server, where $\sum_{i \in \mathcal{N}} \phi_i(t) \mathcal{G}_i(t) \leq \mathcal{O}$; $\boldsymbol{B}(t)$ means battery energy, $\sum_{i=1}^{N} \hat{B}_i(t) \leq \boldsymbol{B}(t)$
- Action $\boldsymbol{A}$: $\boldsymbol{A}(t)$ is equivalent to decision $(p_i(t), \phi_i(t), \psi_i(t), \rho_{i,0,k}^{up}(t), \rho_{i,1,k}^{up}(t), \rho_{i,0,k}^{down}(t), \rho_{i,1,k}^{down}(t))$ at slot time t.
- Reward R: We take the value of $MWD_{total}^{busy}(t)$ as the reward in DQN, due to our aim is minimize the value of $MWD_{total}^{busy}(t)$. Thus, the agent should minimize the reward when learning, so we rewrite **P2** as follows:

$$\mathbf{P2}: \min_{\boldsymbol{V}(t)} \ MWD_{total}^{busy}(t)$$

$$\text{s.t. } (48), (49)$$

In DQN, the transition between states follows Markov Decision Process (MDP). We adopt Q-table $Q(\boldsymbol{S}, \boldsymbol{A})$ to store the reward, and $Q(\boldsymbol{S}(t), \boldsymbol{A}(t))$ means the reward of agent at time slot t. $Q(\boldsymbol{S}(t), \boldsymbol{A}(t))$ is only related the previous time slot $t-1$. However, because the space of state is continuous and the space action is discrete, the storage of $Q(\boldsymbol{S}, \boldsymbol{A})$ will become large. Based on the fact that the state-action set has continuity of values, we use neural network to approximate $Q(\boldsymbol{S}, \boldsymbol{A})$ nonlinearity, that is:

$$Q(\boldsymbol{S}, a; \theta) \approx Q(\boldsymbol{S}, \boldsymbol{A}) \tag{49}$$

where $Q(\boldsymbol{S}, a; \theta)$ means the value are obtained by training θ in neural network.

Algorithm 1 DQN Algorithm for Optimization Problem **P2**

1: Initialize experience replay memory $\mathcal{D}$ to capacity $\mathcal{N}$, step $\leftarrow 0$;
2: Initialize prediction network $Q_\theta(\boldsymbol{S}, \boldsymbol{A})$ with random weights θ;
3: Initialize target network $Q_{\theta^-}(\boldsymbol{S}, \boldsymbol{A})$ with weights $\theta^- = \theta$;
4: **for** episode $e = 1, 2, \ldots$ **do**
5: Initialize the state $\boldsymbol{S} = (\boldsymbol{Pos}(t), \mathcal{O}(t), \boldsymbol{B}(t))$;
6: **for** $t = 1, 2, \ldots, T$ **do**
7: Observe $\boldsymbol{S}(t)$ at time slot t;
8: According probability ϵ to select a random action $\boldsymbol{A}(t)$;
9: Otherwise select action $\boldsymbol{A}(t)$ by $\boldsymbol{A}(t) = \text{argmax}Q(\boldsymbol{S}(t), a(t); \theta)$;
10: Get the next state $\boldsymbol{S}'$ and reward $\boldsymbol{R}$ after executing action $\boldsymbol{A}(t)$;
11: Put the transition tuple ($\boldsymbol{S}$, $\boldsymbol{A}$, $\boldsymbol{R}$, $\boldsymbol{S}'$) into replay memory $\mathcal{D}$;
12: Select randomly minibatch of transitions tuple from $\mathcal{D}$ to train prediction network $Q_\theta(\boldsymbol{S}, \boldsymbol{A})$;
13: Through $\theta_t + \alpha[r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a, \theta)] \nabla Q(s, a; \theta)$ to update the prediction network parameters θ;
14: **if** step ≥ 50 and step $\% 5 = 0$ **then**
15: Set $\theta^- \leftarrow \theta$;
16: **end**
17: Set step $\leftarrow$ step + 1;
18: **end**
19: **end**

For our scenario, we first observe the state $\boldsymbol{S}(t) = (\boldsymbol{Pos}(t), \mathcal{O}(t), \boldsymbol{B}(t))$ at time t. We define a probability ϵ. According to ϵ, the agent can select a decision $\boldsymbol{A}$. Otherwise, the agent can get the decision $\boldsymbol{A}$ according to $Q(\boldsymbol{S}, a; \theta)$

The main feature of DQN is the introduction of experience playback, that is, adding a quadruple ($\boldsymbol{S}$, $\boldsymbol{A}$, $\boldsymbol{R}$, $\boldsymbol{S}'$) to an experience pool, and these quadruples will be used to update $Q(\boldsymbol{S}, \boldsymbol{A}; \theta)$ parameters later.

Finally, according to the loss function, the learnable parameter θ is given as follows:

$$\theta_{t+1} = \theta_t + \alpha \left[r + \gamma \max_{a'} Q\left(s', a'; \theta^-\right) - Q(s, a, \theta) \right] \times \nabla Q(s, a; \theta) \tag{50}$$

We adopt DQN algorithm for optimizing **P2** which can dynamically adapt the change of environment.

Algorithm 1 shows the process of solving problem **P2**. Line 1 initializes replay memory $\mathcal{D}$ to capacity $\mathcal{N}$ and train step. Line 2 initializes prediction network Q with random weights θ. Line 3 initializes target network's weights θ^- with prediction network's weights θ. Line 5 initializes the state $\boldsymbol{S}(t)$ at each episode. In time series of T, time slots are continuous, we take each time slot as a state node. At each time slot t, line 7 shows obtaining observe $\boldsymbol{S}(t)$, lines 8–9 show selecting action $\boldsymbol{A}(t)$ either by random

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

selection according probability ϵ, or by selecting the action of maximum Q value. Line 10 shows the feedback of environment and obtaining the immediate reward $\boldsymbol{R}$ and the next state $\boldsymbol{S}'$ after executing action $\boldsymbol{A}(t)$. Line 11 puts the ($\boldsymbol{S}$, $\boldsymbol{A}$, $\boldsymbol{R}$, $\boldsymbol{S}'$) into replay memory $\mathcal{D}$ for learning. Lines 12–13 show that we select randomly minibatch of transitions tuple from replay memory $\mathcal{D}$, and train the prediction network and update it's weights θ. Lines 14–16 show updating the target network's weights θ^- by prediction network's weights θ when training step meet the conditions. Finally, seting step+1 in line 17.

C. Algorithm Complexity for the Two Algorithms

The computational complexity of SAGA depends on three parts, namely, initialization, SA algorithm, GA algorithm and fitness value calculation. We will analyze it respectively.

1) As $2r$ search agents, each search agent evaluates the fitness value, so the computational complexity of the initialization process is $O(2r \times D_1)$, where D_1 represents the variable dimension of **P3**.
2) The computational complexity of SA is $O(L \times Max_iter1 \times D_1)$, among L is the number of Inner cycles for each T, Max_iter1 represents the maximum number of iterations required for the algorithm to terminate, and D_1 represents the variable dimension of **P3**. Because the algorithm termination condition is $T > T_{final}$ and each iteration satisfying $T = \omega T$. Then we can obtain $(\omega^\wedge Max_iter1)T \leq T_{final}$, so we can derive $Max_iter1 \leq (T_{final}/T)^\wedge(1/\omega)$. Therefore, we can obtain the total complexity of SA is $O(L \times (T_{final}/T)^\wedge(1/\omega) \times D_1)$.
3) Because r pairs of individuals are required to cross and mutate for each time, and it generates r pairs new individuals, and each pair needs to be fitted. Since GA is annealed in SA cycle, the complexity of GA process is $O(L \times (T_{final}/T)^\wedge(1/\omega) \times 2r \times D_1)$.

In summary, the total computational complexity of SAGA is

$$O\left(2r \times D_1 + L \times (T_{final}/T)^\wedge (1/\omega) \times D_1 + L \times (T_{final}/T)^\wedge (1/\omega) \times 2r \times D_1\right). \quad (51)$$

Next we will propose the algorithm complexity of DQN. In algorithm of DQN, we assume that the running time of the Line 1 is t_0, the running time of the Line 2 is t_1, and the running time of the Line 3 is t_2. The average running time of the Line 4 is t_3 with M repetitions; the running time of the Line 5 is $t_{3.1}$; the average running time of the Line 6 is $t_{3.2}$ with T repetitions; the running time of the Line 7 is $t_{3.2.1}$. The running time of Line 8 to 9 is $t_{3.2.2}$, Line 10 is $t_{3.2.3}$, Line 11 is $t_{3.2.4}$, Line 12 is $t_{3.2.5}$, Line 13 is $t_{3.2.6}$, Line 14 to 16 is $t_{3.2.7}$, and Line 17 is $t_{3.2.8}$.

According to the average time assumption of execution on the code, the time of execution of DQN algorithm can be calculated as:

$$\begin{aligned} T(episode, t) &= t_{c1} + (t_{c2} + t_{c3} * T) * M \\ &= t_{c1} + (t_{c2} * M + t_{c3} * T * M) \\ &= t_{c3} * T * M \\ &= T * M \end{aligned} \quad (52)$$

The constant terms are combined in the formula deduction. When the values of $episode$ and t are very large, the influence of the constants in the function $T(episode, t)$ and the coefficients of T and M on $episode$ and t can also be ignored. Meanwhile, we note that the main influencing factor of $T(episode, t)$ function is $T * M$ rather than M, because $T * M$ grows much faster than M. Therefore, the time complexity of function $T(episode, t)$ can be expressed as

$$T(episode, t) = O(n_t n_m) \quad (53)$$

where n_t refers to the number of time steps in each $episode$, and n_m refers to the number of $episode$.

TABLE II
SIMULATION PARAMETERS

Notations	Reference value
r	100
L	1000
$\hbar$	0.6
γ	0.02
ϖ	0.01
T_{final}	0
τ	1
β_i	5kB
σ^2	0.05
V	1000
ζ	0.01
f_i	2GHz
T	100
$EPOCH$	50
N_{SBS}	10
Lr	0.01
$reward_{decay}$	0.95
e_{greedy}	0.9

VII. PERFORMANCE EVALUATIONS

In this part, we conduct simulations to evaluate the proposed resource allocation and computation offloading scheme of MEC enabled IoV network. And we simulate with SAGA and DQN in the off-peak hours and peak hours respectively due to our proposed model contains two periods. The parameters are given as follows in Table II.

A. The Situation of the Off-Peak Hours

Firstly, we study the influence of caching or not on the average execution delay in the off-peak hours. As can be seen in Fig. 3, with the increase of iteration numbers, the cached and non-cached curves converge gradually, and then reach a stable state after roughly 5 iterations, the average execution delay no longer fluctuates greatly with the increase of iteration numbers. From the stable curve, it can be seen that MEC with caching has more advantages in delay than MEC without caching. However,

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

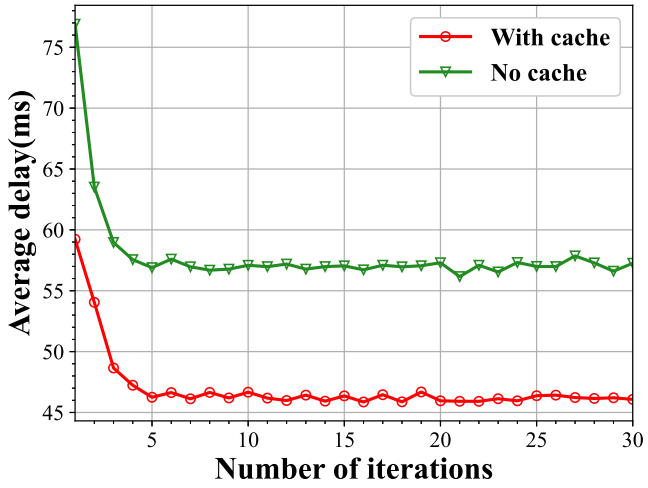


Fig. 3. Average delay vs. number of iterations under no caching and with caching in the off-peak hours.

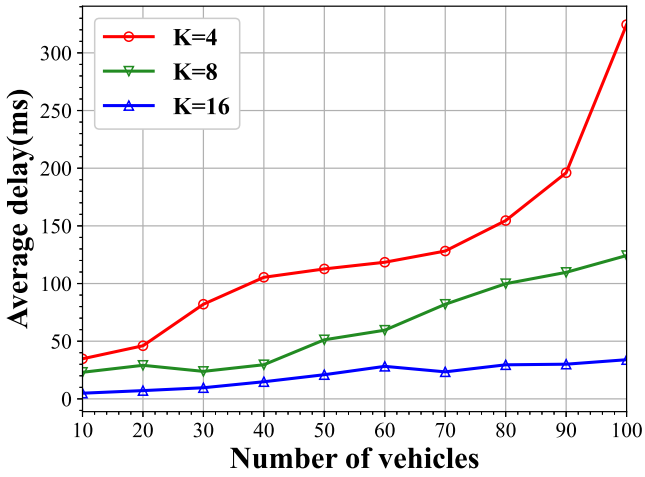


Fig. 5. Average delay vs. number of vehicles under different number of channels in the off-peak hours.

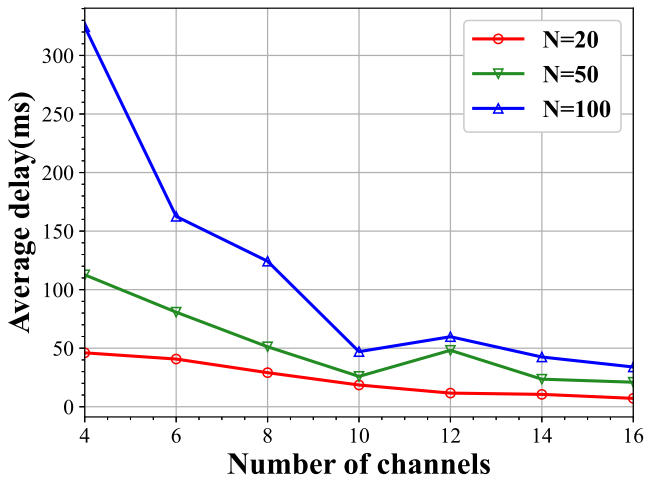


Fig. 4. Average delay vs. number of channels under different number of vehicles in the off-peak hours.

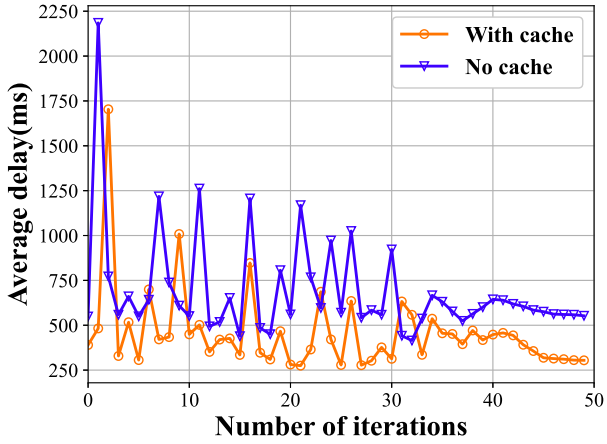


Fig. 6. Average delay vs. number of iterations under no caching and with caching in peak hours.

due to the off-peak hours, the transmission resources and computing resources in the IoV are abundant, and the average delay difference between MEC with caching and without caching is very small.

Next, we study the influence of the number of idle channels on the average execution delay under different number of vehicles, as shown in Fig. 4. It can be seen from the curve that when the number of vehicles is fixed, with the increase of channel numbers, the overall average execution delay has a downward trend. In vertical view, with the increase of vehicle numbers, the average execution delay also increases. When the number of channels is small, the gap between them is larger, and when the number of channels is large, the gap between them is smaller.

As shown in the Fig. 5, we study the influence of the number of idle vehicles on the average execution delay under different channel numbers. It can be seen that when the number of channels is fixed, with the increase of the number of vehicles, the average execution delay has an overall upward trend, in which the curve of $K = 4$ has the most obvious increasing, followed by $K = 8$, and $K = 16$ is very gentle. On the other hand, when the number of vehicles is fixed, the average execution delay increases with the increase of channel numbers. When the number of vehicles is small, the average delay gap among the three is very small. When the number of vehicles is large, resulting in the shortage of wireless subchannel resources and MEC computation resources, as more and more vehicles in the IoV compete the limited wireless and computation resource to offload, the average delay gap among the three is rapidly widened, and the smaller the number of channels, the greater the average execution delay.

B. The Situation of the Peak Hours

Firstly, we study the influence of caching or not on the average execution delay in the peak hours. It can be seen from Fig. 6, when the curve has not converged, the average execution delay with or without caching fluctuates greatly, but the average execution delay without caching is larger than that with caching. With

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

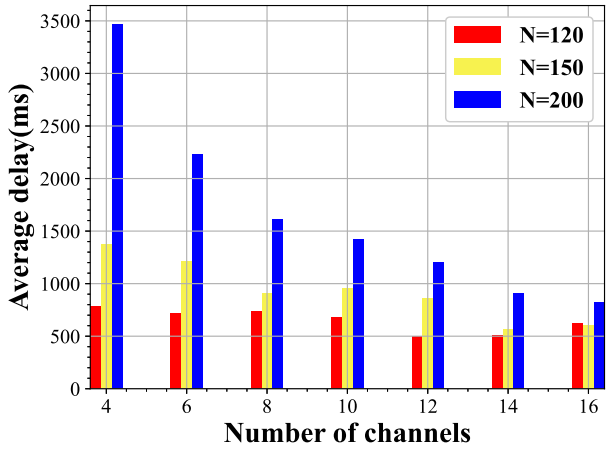


Fig. 7. Average delay vs. number of channels under different number of vehicles in peak hours.

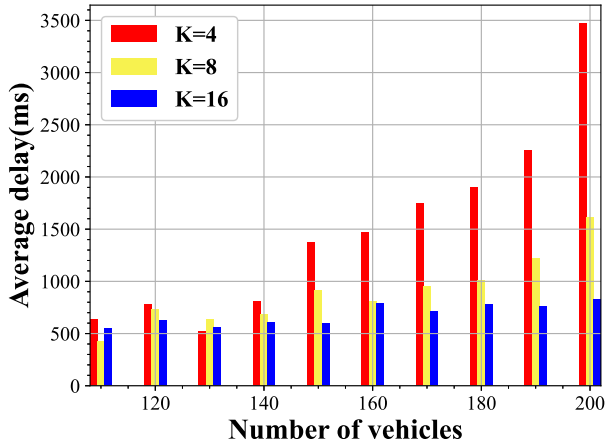


Fig. 8. Average delay vs. number of vehicles under different number of channels in peak hours.

the increase of iteration numbers, the cached and non-cached curves converge gradually, and it reaches a stable state after 43 iterations. The average execution delay no longer continues to jitter with the increase of iteration times. As can be seen from the stabilized curve, MEC with caching has more advantages in latency than MEC without caching. However, because we define the rate between MEC server and cloud center is very high in the simulation, the data packets required for tasks are not very large, so it does not take a long time for the data cached in the cloud to be transmitted back to MEC server. Therefore, it can be seen that the average execution delay difference between cached and no caching is about 250 ms, which is not particularly large.

Then, we compare the influence of the number of busy channels on the average execution delay under different number of vehicles, as shown in Fig. 7. It can be seen from the curve that when the number of vehicles is fixed, with the increase of channel numbers, the average execution delay has an overall downward trend. On the other hand, when the number of channels is fixed, the average execution delay increases with the increase of vehicle numbers. And when the number of channels is small, the average delay gap among the three is obvious; when the number of channels is large, the average delay gap among the three is extremely small due to sufficient computing resources and channel resources.

Finally, we compare the influence of the number of busy vehicles on the average execution delay under different channel numbers, as shown in the Fig. 8. It can be seen from the curve that when the number of channels is fixed, the average execution delay is gentle and fluctuates little between the number of vehicles 110 and 140, but after the number of vehicles is greater than 140, with the increase of the number of vehicles, the average execution delay has an overall upward trend, among which the curve $K = 4$ has the most obvious increasing, and $K = 16$ is very gentle. On the other hand, when the number of vehicles is fixed between 110 and 140, the average execution delay gap is not large with the increase of channel numbers. And when the number of vehicles is greater than 140, due to the imbalance between supply and demand of computing resources and channel resources, the average execution delay gap between the three is rapidly widened, and the smaller the number of channels, the larger the average execution delay.

VIII. Conclusion

In this article, we have developed the task offloading and resource allocation scheme for the cases of off-peak hours and peak hours in a MEC enabled IoV system with a number of energy harvesting vehicles. In off-peak hours, firstly, by taking the advantage of the Lyapunov optimization, we transform the original problem as the corresponding upper bound of the Lyapunov drift-plus-penalty function and propose a dynamic algorithm over consecutive time slots; secondly, for a specific time slot, we derive the optimal solution with SAGA. In peak hours, the optimized variable not only including the strategies of data transmission path, the subchannels assignment, the caching and caching updating, but also the decision of execution mode, we adopt the DQN algorithm to solve the constructed integer programming problem, which can achieve imitation learning efficiently. Finally, performance evaluations are conducted to verify the effectiveness and superior of the proposed schemes. In future research, we plan to combine the blockchain with task offloading in IoV with the purpose to improve the efficiency of computation offloading and the security of data transmission.

References

[1] S. Yu, X. Gong, Q. Shi, X. Wang, and X. Chen, "EC-SAGINs: Edge computing-enhanced space-air-ground integrated networks for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5742–5754, Apr. 2022, doi: 10.1109/JIOT.2021.3052542.

[2] X. He, H. Lu, M. Du, Y. Mao, and K. Wang, "QoE-based task offloading with deep reinforcement learning in edge-enabled Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2252–2261, Apr. 2021.

[3] X. Yuan, J. Chen, N. Zhang, J. Ni, F. R. Yu, and V. C. M. Leung, "Digital twin-driven vehicular task offloading and IRS configuration in the Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24290–24304, Dec. 2022.

[4] S. Malektaji, A. Ebrahimzadeh, H. Elbiaze, R. H. Glitho, and S. Kianpisheh, "Deep reinforcement learning-based content migration for edge content delivery networks with vehicular nodes," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 3415–3431, Sep. 2021.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.

[5] J. Liu, W. Wang, D. Li, S. Wan, and H. Liu, "Role of gifts in decision making: An endowment effect incentive mechanism for offloading in the IoV," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6933–6951, Aug. 2019.
[6] J. Wu et al., "Resource allocation for delay-sensitive vehicle-to-multi-edges (V2Es) communications in vehicular networks: A multi-agent deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1873–1886, Apr.–Jun. 2021.
[7] X. Ren et al., "Ai-Bazaar: A cloud-edge computing power trading framework for ubiquitous AI services," *IEEE Trans. Cloud Comput.*, early access, Aug. 25, 2022, doi: 10.1109/TCC.2022.3201544.
[8] T. Ouyang, X. Chen, Z. Zhou, R. Li, and X. Tang, "Adaptive user-managed service placement for mobile edge computing via contextual multi-armed bandit learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1313–1326, Mar. 2023.
[9] R. Li, Z. Zhou, X. Zhang, and X. Chen, "Joint application placement and request routing optimization for dynamic edge computing service management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4581–4596, Dec. 2022.
[10] W. Quan, M. Liu, N. Cheng, X. Zhang, D. Gao, and H. Zhang, "Cybertwin-driven DRL-based adaptive transmission scheduling for software defined vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 4607–4619, May 2022.
[11] J. Zhao, Q. Li, Y. Gong, and K. Zhang, "Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 7944–7956, Aug. 2019.
[12] Z. Ning et al., "Deep reinforcement learning for intelligent Internet of Vehicles: An energy-efficient computational offloading scheme," *IEEE Trans. Cogn. Commun. Netw.*, vol. 5, no. 4, pp. 1060–1072, Dec. 2019.
[13] Z. Chang, L. Liu, X. Guo, and Q. Sheng, "Dynamic resource allocation and computation offloading for IoT fog computing system," *IEEE Trans. Ind. Informat.*, vol. 17, no. 5, pp. 3348–3357, May 2021.
[14] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14198–14211, Dec. 2020.
[15] H. Liang et al., "Reinforcement learning enabled dynamic resource allocation in the Internet of Vehicles," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4957–4967, Jul. 2021.
[16] Y. Chen, N. Zhang, Y. Zhang, X. Chen, W. Wu, and X. S. Shen, "TOFFEE: Task offloading and frequency scaling for energy efficiency of mobile devices in mobile edge computing," *IEEE Trans. Cloud Comput.*, vol. 9, no. 4, pp. 1634–1644, Oct.–Dec. 2021.
[17] Y. Ju et al., "Joint secure offloading and resource allocation for vehicular edge computing network: A multi-agent deep reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 5, pp. 5555–5569, May 2023.
[18] Z. Li, M. Xu, J. Nie, J. Kang, W. Chen, and S. Xie, "NOMA-enabled cooperative computation offloading for blockchain-empowered Internet of Things: A learning approach," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2364–2378, Feb. 2021.
[19] S. Yang, J. Liu, F. Zhang, F. Li, X. Chen, and X. Fu, "Caching-enabled computation offloading in multi-region MEC network via deep reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21086–21098, Nov. 2022.
[20] C. Wang, C. Chen, Q. Pei, N. Lv, and H. Song, "Popularity incentive caching for vehicular named data networking," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 4, pp. 3640–3653, Apr. 2022.
[21] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Collaborative data scheduling for vehicular edge computing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9637–9650, Oct. 2020.
[22] Q. Tang, R. Xie, T. Huang, W. Feng, and Y. Liu, "Dynamic computation offloading with imperfect state information in energy harvesting small cell networks: A partially observable stochastic game," *IEEE Wireless Commun. Lett.*, vol. 9, no. 8, pp. 1300–1304, Aug. 2020.
[23] Z. Wei, B. Zhao, J. Su, and X. Lu, "Dynamic edge computation offloading for Internet of Things with energy harvesting: A learning method," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4436–4447, Jun. 2019.
[24] S.-S. Lee and S. Lee, "Resource allocation for vehicular fog computing using reinforcement learning combined with heuristic information," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10450–10464, Oct. 2020.
[25] X. Zhang, M. Peng, S. Yan, and Y. Sun, "Deep-reinforcement-learning-based mode selection and resource allocation for cellular V2X communications," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6380–6391, Jul. 2020.
[26] X. Qi, J. Chong, Q. Zhang, and Z. Yang, "Collaborative computation offloading in the multi-UAV fleeted mobile edge computing network via connected dominating set," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10832–10848, Oct. 2022.
[27] M. Z. Alam, F. S. Abkenar, I. Adhicandra, S. Murali, and A. Jamalipour, "Low-delay path selection for cluster-based buffer-aided vehicular communications," *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 9356–9363, Sep. 2020.
[28] L.-H. Yen, J.-C. Hu, Y.-D. Lin, and B. Kar, "Decentralized configuration protocols for low-cost offloading from multiple edges to multiple vehicular fogs," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 872–885, Jan. 2021.
[29] Z. Wang, D. Zhao, M. Ni, L. Li, and C. Li, "Collaborative mobile computation offloading to vehicle-based cloudlets," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 768–781, Jan. 2021.
[30] B. Liu, W. Zhang, W. Chen, H. Huang, and S. Guo, "Online computation offloading and traffic routing for UAV swarms in edge-cloud computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8777–8791, Aug. 2020.
[31] Z. Ning et al., "Intelligent edge computing in Internet of Vehicles: A joint computation offloading and caching solution," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2212–2225, Apr. 2021.
[32] L. Vigneri, T. Spyropoulos, and C. Barakat, "Quality of experience-aware mobile edge caching through a vehicular cloud," in *Proc. 20th ACM Int. Conf. Model., Anal. Simul. Wireless Mobile Syst.*, 2017, pp. 91–98. [Online]. Available: https://doi.org/10.1145/3127540.3127555
[33] M. A. Javed and S. Zeadally, "AI-empowered content caching in vehicular edge computing: Opportunities and challenges," *IEEE Netw.*, vol. 35, no. 3, pp. 109–115, May/Jun. 2021.
[34] C. Chen, C. Wang, T. Qiu, M. Atiquzzaman, and D. O. Wu, "Caching in vehicular named data networking: Architecture, schemes and future directions," *IEEE Commun. Surv. Tut.*, vol. 22, no. 4, pp. 2378–2407, Fourthquarter 2020.
[35] G. Qiao, S. Leng, S. Maharjan, Y. Zhang, and N. Ansari, "Deep reinforcement learning for cooperative content caching in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 247–257, Jan. 2020.
[36] H. Tian et al., "CoPace: Edge computation offloading and caching for self-driving with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13281–13293, Dec. 2021.
[37] Y. Dong, S. Guo, Q. Wang, S. Yu, and Y. Yang, "Content caching-enhanced computation offloading in mobile edge service networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 872–886, Jan. 2022.
[38] S. Chen, L. Rui, Z. Gao, W. Li, and X. Qiu, "Cache-assisted collaborative task offloading and resource allocation strategy: A metareinforcement learning approach," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19823–19842, Oct. 2022.
[39] W. Hao and Z. Hongfu, "Using genetic annealing simulated annealing algorithm to solve disassembly sequence planning," *J. Syst. Eng. Electron.*, vol. 20, no. 4, pp. 906–912, Aug. 2009.
[40] T. Renyuan, Y. Shiyou, L. Yan, W. Geng, and M. Tiemin, "Combined strategy of improved simulated annealing and genetic algorithm for inverse problem," *IEEE Trans. Magn.*, vol. 32, no. 3, pp. 1326–1329, May 1996.
[41] Y. C. Wu, T. Q. Dinh, Y. Fu, C. Lin, and T. Q. S. Quek, "A hybrid and optimization approach for strategy and resource allocation in networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4282–4295, Jul. 2021.
[42] J. Martín-Pérez, K. Antevski, A. Garcia-Saavedra, X. Li, and C. J. Bernardos, "DQN dynamic pricing and revenue driven service federation strategy," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 3987–4001, Dec. 2021.
[43] J. Pan, X. Wang, Y. Cheng, and Q. Yu, "Multisource transfer double DQN based on actor learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 6, pp. 2227–2238, Jun. 2018.



Liqing Liu received the master's degree from the College of Science, Yanshan University, Qinhuangdao, China, in 2015 and the Ph.D. degree from the College of Information Science and Engineering, Yanshan University, in 2019. Since 2019, she has been a Teacher with the School of Computer and Communication Engineering, Northeastern University, Shenyang, China. Her research interests include computing offloading, cloud resource allocation, and Internet of Vehicles.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.



Xiaoming Yuan (Member, IEEE) received the B.E. degree in electronics and information engineering from Henan Polytechnic University, Jiaozuo, China, in 2012 and the Ph.D. degree in communication and information system from Xidian University, Xi'an, China, in 2018. She is currently an Assistant Professor of Qinhuangdao Branch Campus, Northeastern University, Shenyang, China. Her research interests include cloud/edge computing, artificial intelligence, and the Internet of Vehicles.



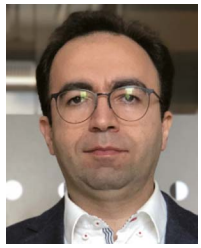
Ning Zhang (Senior Member, IEEE) received the Ph.D degree in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada, in 2015. He is currently an Associate Professor and Canada Research Chair with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON, Canada. After that, he was a Postdoc research Fellow with the University of Waterloo, Waterloo, ON, Canada, and University of Toronto, Toronto, ON, Canada, respectively. His research interests include connected vehicles, mobile edge computing, wireless networking, and security. He is a Distinguished Lecturer of IEEE ComSoc, a Highly Cited Researcher (Web of Science), as well as the Vice Chair of IEEE Technical Committee on Cognitive Networks and IEEE Technical Committee on Big Data. He is/was an Associate Editor for IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, IEEE INTERNET OF THINGS JOURNAL, and IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS and Networking. He is/was a TPC/General Chair of numerous conferences and workshops, such as IEEE ICC, VTC, INFOCOM Workshop, and Mobicom Workshop. He was the recipient of the several best paper awards from conferences and journals, such as IEEE Globecom, IEEE ICC, IEEE ICCC, IEEE WCSP, and Journal of Communications and Information Networks.



Decheng Chen received the B.E. degree in computer science and technology from the China University of Petroleum, Beijing, China. Since September 2020, he has been working toward the master's degree in computer technology with Qinhuangdao Branch Campus, Northeastern University, Shenyang, China. His research interests include cloud/edge computing and Internet of Vehicles.



Keping Yu (Member, IEEE) received the M.E. and Ph.D. degrees from the Graduate School of Global Information and Telecommunication Studies, Waseda University, Tokyo, Japan, in 2012 and 2016, respectively. He was a Research Associate, Junior Researcher and Researcher with the Global Information and Telecommunication Institute, Waseda University, Tokyo, Japan, from 2015 to 2019, 2019 to 2020, and 2020 to 2022, respectively. He is currently an Associate Professor with Hosei University, Tokyo, and a Visiting Scientist with the RIKEN Center for Advanced Intelligence Project, Tokyo. He has authored more than 200 peer-review research papers and books, including over 60 IEEE/ACM Transactions papers. His research interests include smart grids, information-centric networking, the Internet of Things, artificial intelligence, blockchain, and information security.



Amir Taherkordi received the Ph.D. degree from Informatics Department, University of Oslo (UiO), Oslo, Norway, in 2011. He is currently a Full Professor with the Department of Informatics, UiO. He joined Sonitor Technologies as a Senior Embedded Software Engineer. From 2013 to 2018, he was a Researcher with the Networks and Distributed Systems Group, Department of Informatics, UiO. He has authored or coauthored several articles in high-ranked conferences and journals, and he has experience from several national (Norwegian Research Council) and international (European research funding agencies) research projects. His research interests include resource-efficiency, scalability, adaptability, dependability, mobility and data-intensiveness of distributed systems designed for emerging computing technologies, such as Internet of Things, Fog/Edge/Cloud Computing, and Cyber-Physical Systems. He is an Associate Editor for IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING.

Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 02,2026 at 02:45:55 UTC from IEEE Xplore. Restrictions apply.